

面向对象程序设计基础 (OOP)

黄民烈

aihuang@tsinghua.edu.cn

<https://coai.cs.tsinghua.edu.cn/html>

课程团队：刘知远 任炬 黄民烈

上期要点回顾

- 函数重载(名相同而义不同)
- 自定义类与对象(封装)
- 数据成员、成员函数(封装=数据+服务)
- 访问权限(隐藏实现+提供服务)
- this指针
- 宏与内联函数

本讲内容提要

- 4.1 构造函数
- 4.2 析构函数
- 4.3 对象的构造与析构时机(局部对象和全局对象)
- 4.4 引用
- 4.5 运算符重载

如何使含有对象的程序更可靠？

■ OOP三性：简单性、清晰性、普遍性

■ 对用户定义类型进行严格的类型检查

■ 隐藏实现，防止受到不必要的干扰

- 由于隐藏实现（访问权限控制），对象的有些私有数据成员只有类的设计者知道，而且只允许类的成员函数访问。

■ 对象的初始化和清除，需要自动进行

- 忘记初始化或清除变量可能会导致程序崩溃。
- 由类生成的对象是一种新型的变量，也要初始化。
- 尽管可以由通过显式调用对象成员函数来初始化对象，但这种做法缺少强制性，因而容易被程序员遗忘。

如何使含有对象的程序更可靠？

■ 结论

- 如何进行初始化和清除(HOW)，应由类设计者决定。
- 何时进行初始化和清除(WHEN)，应由编译器来决定。

构造函数：对象的“生”

- 对象的“生”（初始化工作）是由编译器在创建对象处自动生成调用构造函数的代码来完成的。
- 构造函数是类的特殊成员函数，它用来确保类的每个对象都能正确地初始化。

构造函数

- 构造函数没有返回值类型，函数名与类名相同
- 类的构造函数可以重载，即可以使用不同的函数参数进行对象初始化

```
class Student {  
    int ID;  
public:  
    Student(int id) { ID = id; }  
    Student(int year, int order) {  
        ID = year * 10000 + order;  
    }  
    ...  
};
```

构造函数的初始化列表

- 构造函数可以使用初始化列表**初始化成员数据**
- 该列表在定义构造函数时使用，位置在函数体之前、函数参数列表之后，以冒号作开头。
- 使用“数据成员(初始值)”的形式

```
class Student {  
    int ID;//声明  
public:  
    Student(int id) : ID(id) { }  
    Student(int year, int order) {  
        ID = year * 10000 + order;  
    }  
    ...  
};
```

构造函数的初始化列表

- 初始化列表的成员是按照**声明的顺序**初始化的，而不是按照出现在初始化列表中的顺序
- 在下面的代码中，编译器先初始化**ID1**，再初始化**ID2**，因此**ID1**的值将不可预测

```
class Student {  
    int ID1; //声明  
    int ID2; //声明  
public:  
    Student(int id) : ID2(id), ID1(ID2) { }  
    ...  
};
```

构造函数的初始化列表

- 在构造函数的初始化列表中，还可以调用其他构造函数，称为“委派构造函数”

```
class Info {  
public:  
    Info() { Init(); }  
    Info(int i) : Info() { id = i; }  
    Info(char c) : Info() { gender = c; }  
private:  
    void Init() { .... } // 其他初始化  
    int id;  
    char gender;  
    ...  
};
```

构造函数

■ 就地初始化

- C++11之前，类中的一般成员变量不能在类定义时进行初始化，它们的初始化操作只能通过构造函数进行。
- C++11新增支持如下初始化操作，称为就地初始化：

```
class A {  
    private:  
        int a = 1; //声明+初始化  
        double b {2.0}; //声明+初始化  
    public:  
        A() {} //a=1 b=2.0  
        A(int i):a(i) {} //a=i b=2.0  
        A(int i, double j):a(i), b(j) {} //a=i b=j  
};
```

注意：就地初始化只是一种简便的表达方式，实际操作仍然在对象构造的时候执行。

默认构造函数

- 不带任何参数的构造函数，或每个形参提供默认实参的构造函数，被称为“默认构造函数”，也称“缺省构造函数”

```
class A {  
    private:  
        int a = 1;  
        double b {2.0};  
    public:  
        A() {} //定义默认构造函数  
        A(int i):a(i) {}  
        A(int i, double j):a(i), b(j) {}  
};
```

默认构造函数

- 使用默认构造函数（没有参数）来生成对象时，对象定义的格式为：

```
ClassName a;    //调用默认构造函数  
ClassName b = ClassName();  
                //同样调用默认构造函数
```

注意区分：

```
ClassName c();  
    //这声明了一个返回值为ClassName的函数
```

默认构造函数

- 在类的构造函数中，除了执行函数体内声明的语句，编译器还会做一些额外操作

```
#include <iostream>
```

```
using namespace std;
```

```
class Member {
```

```
public:
```

```
    Member() { cout << "Member()" << endl; }
```

```
};
```

```
class Test {
```

```
public:
```

```
    Member m;
```

```
    Test() { cout << "Test()" << endl; }
```

```
};
```

```
Test t;
```

```
int main() { return 0; }
```

输出：

Member()
Test()

- 例如会自动调用成员变量的默认构造函数

- 先调用成员变量的构造，再执行自己的构造函数

默认构造函数

■ 隐式定义的默认构造函数

- 有时候我们没有手动定义默认构造函数，但我们仍然能够按上述方式定义类的对象
- 这是因为编译器帮我们**隐式地合成**了一个默认构造函数

```
class A {  
public:  
    int data = 0;  
};  
  
A a;
```

等价于

```
class A {  
public:  
    int data = 0;  
    A() {}  
};  
  
A a;
```

默认构造函数

■ 隐式定义的默认构造函数

- 若用户已经定义了其他构造函数，编译器将不会隐式合成默认构造函数

```
class Student {  
  private:  
    int ID = 1;  
  public:  
    Student(int i):ID(i) {}  
};
```

```
Student s; //编译错误
```

默认构造函数

■ 显式声明默认构造函数

- 出于某些需要，我们可以**手动**指定生成默认版本的构造函数：即便其他构造函数存在，编译器也会定义隐式默认构造函数

```
class Student {  
    private:  
        int ID = 1;  
    public:  
        Student() = default;    // C++11起  
        Student(int i):ID(i) {}  
};  
  
Student s;
```

默认构造函数

■ 显式删除构造函数

- 有时候，我们需要显式地声明禁用某种构造函数。
- 如果我们定义类如下，会出现什么问题？

```
class Student {  
private:  
    int ID = 1;  
    char role = 'a';  
public:  
    Student() = default;  
    Student(int i):ID(i) {}  
};
```

```
Student s('c');
```

默认构造函数

■ 显式删除构造函数

- 按照下面方法生成类对象，编译和执行都不会报错。
- 此时 **'c'** 先被转换成 **int** 型值，然后调用构造函数 **Student(int i)**。

```
class Student {  
private:  
    int ID = 1;  
    char role = 'a';  
public:  
    Student() = default;  
    Student(int i):ID(i) {}  
};
```

```
Student s('c');
```

默认构造函数

■ 显式删除构造函数

- 从正确性上讲，这样的代码没有问题，但是从工程的角度讲，这是很危险的行为。因为在开发者看来，用字符初始化应该是未定义的行为。

```
class Student {  
private:  
    int ID = 1;  
    char role = 'a';  
public:  
    Student() = default;  
    Student(int i):ID(i) {}  
};
```

```
Student s('c');
```

默认构造函数

■ 显式删除构造函数

- 使用delete显式地删除构造函数，避免产生未预期行为的可能性。

```
class Student {  
    private:  
        int ID = 1;  
        char role = 'a';  
    public:  
        Student() = default;  
        Student(int i):ID(i) {}  
        Student(char cls) = delete;  
};
```

```
Student s('c'); // 编译错误
```

默认构造函数

■ 默认构造函数

- 默认构造函数在什么情况下会被隐式定义?
- 默认构造函数的行为?
- 参考

https://zh.cppreference.com/w/cpp/language/default_constructor

- ## ■ 隐式定义的默认构造函数还常出现在继承和虚函数的问题中（以后内容）

对象数组的初始化

- 无参定义对象数组，必须要有默认构造函数

`A a[50];` // 定义了一个具有50个元素的A类对象数组

- 如果构造函数只有一个参数

`A a[3] = {1, 3, 5};` // 三个实参分别传递给3个数组元素的构造函数

- 如果构造函数有多个参数

`A a[3] = {A(1, 2), A(3, 5), A(0, 7)};` // 构造函数有两个整型参数

析构函数：对象的“死”

- 对象的“死”（清除和释放资源）是由编译器在对象作用域结束处自动生成调用析构函数代码来完成的。
 - 当执行到“包含对象定义范围结束处”时，编译器自动调用对象的析构函数。
 - 动态分配的内存是一种典型的需要释放的资源。
- 清除对象占用的资源是无条件的，不需要任何选项。因此，析构函数没有参数，且只有一个（即清除方式唯一）。

析构函数

- 一个类只有一个析构函数，名称是“~类名”，**没有函数返回值，没有函数参数。**
- 编译器在对象生命期结束时自动调用类的析构函数，以便释放对象占用的资源，或其他后处理

```
class Classroom {  
    int num;  
    int* ID_list;  
public:  
    Classroom() : num(0), ID_list(nullptr) {}  
    ...  
    ~Classroom() { // 析构函数  
        if (ID_list) delete[] ID_list; // 释放内存  
    }  
};
```

析构函数

- 和默认构造函数一样，析构函数除了执行函数体内声明的语句，编译器还会做一些额外操作

```
#include <iostream>
using namespace std;
class Member {
public:
    ~Member() { cout << "~Member()" << endl; }
};
class Test {
public:
    Member m;
    ~Test() { cout << "~Test()" << endl;}
};
Test t;
int main() { return 0; }
```

输出：

```
~Test()
~Member()
```

- 例如会自动调用成员变量的析构函数

- 先执行自己的析构函数，再调用成员变量的析构

析构函数

■ 隐式定义的析构函数

- 和构造函数类似，当用户没有自定义析构函数时，编译器会自动合成一个隐式的析构函数

```
class Classroom {  
    int num;  
    int* ID_list;  
};
```

等价于

```
class Classroom {  
    int num;  
    int* ID_list;  
public:  
    ~ClassRoom() {}  
};
```

■ 注意隐式定义的析构函数不会delete指针成员

- 因此上述例子可能造成内存泄露

析构函数

■ 析构函数

- 析构函数在什么情况下会被隐式定义?
- 析构函数的行为?
- 参考

<https://zh.cppreference.com/w/cpp/language/destructor>

- 除了用户自定义的代码，析构函数还将自动拓展一些行为，之后会在继承、虚函数的部分介绍（以后内容）

局部对象的构造与析构

■ 局部对象

- 在程序执行到该局部对象的代码时被初始化。
- 在局部对象生命周期结束、即所在作用域结束后被析构。

■ 作用域

- 该变量能够引用的区域
- 例如，`{}`将会形成一个作用域

局部对象的构造与析构

```
#include <iostream>
using namespace std;
class Example {
    int index;
public:
    Example(int i): index(i)
        {cout << index << " is created\n"; }
    ~Example() { cout << index << " is destroyed\n"; }
};

void create_example(int i) {
    Example e(i); // 只在函数内存在
    cout << "Function is over\n";
}

int main() {
    for(int i = 1; i < 3; i++) {
        Example e(0); // 只在当前循环内存在
        create_example(i);
    }
    return 0;
}
```

0 is created
1 is created
Function is over
1 is destroyed
0 is destroyed
0 is created
2 is created
Function is over
2 is destroyed
0 is destroyed

全局对象的构造与析构

■ 全局对象

- 在main()函数调用之前进行初始化。
- 在同一编译单元中，按照**定义顺序**进行初始化。
 - 编译单元：通常同一编译单元就是同一源文件。
- **不同编译单元中，对象初始化顺序不确定。**
- 在main()函数执行完return之后，对象被析构。

全局对象的构造与析构

■ 尽量少用全局对象

- 全局对象的构造顺序不能完全确定，所以全局对象之间不能有依赖关系，否则会出现问题
- 全局对象会增大代码的耦合性，导致程序难以复用或者测试
- 使用参数来替代全局对象

```
void foo() {  
    input.doSomething();  
}  
Input input;  
int main() {  
    foo();  
}
```



```
void foo(Input input) {  
    input.doSomething();  
}  
int main() {  
    Input input;  
    foo(input);  
}
```

引用

■ 具名变量的别名： 类型名 & 引用名 变量名

例： `int v0; int & v1 = v0;` `v1`是变量`v0`的引用，它们在内存中是同一单元的两个不同名字

■ 引用必须在定义时进行初始化，且不能修改引用指向

■ 被引用变量名可以是类的成员变量，如
`int & m = s.m;`

引用

■ 创建引用

```
#include <iostream>
using namespace std;
int main() {
    int i = 1;
    cout << "i=" << i << endl; // i=1
    int& j = i; // j是初始化为i的int引用
                // i和j是同一个变量的两个别名
    cout << "j=" << j << endl; // j=1
    i = 2;
    cout << "j=" << j << endl; // j=2, 修改i, 等于修改j
    j = 3;
    cout << "i=" << i << endl; // i=3, 修改j, 等于修改i
    return 0;
}
```

引用

- 函数参数可以是引用类型，表示函数的形式参数与实际参数是同一个变量，改变形参将改变实参。如调用以下函数将交换实参的值：

```
void swap(int& a, int& b)
{ int tmp = b; b = a; a = tmp; }
```

- 函数返回值可以是引用类型，但不得指向函数的临时变量

比较：参数中的值、引用

```
void swap(int a, int b)
{
    int tmp = a;
    a = b;
    b = tmp;
}

swap(a, b);
```

```
void swap(int *a, int *b)
{
    int tmp = *a;
    *a = *b;
    *b = tmp;
}

swap(&a, &b);
```

```
void swap(int &a, int &b)
{
    int tmp = a;
    a = b;
    b = tmp;
}

swap(a, b);
```

引用

■把引用作为返回值

```
#include <iostream>
using namespace std;
```

```
int a[3] = {1,3,5}; // 全局数组
int& get(int i) { return a[i]; } // 返回a[i]的引用
int main() {
    for(int i = 0; i < 3; i++) {
        cout << "old a[" << i << "]= " << get(i) << endl;
        get(i) += 1;
        cout << "new a[" << i << "]= " << get(i) << endl;
    }
    return 0;
}
```

```
old a[0]=1
new a[0]=2
old a[1]=3
new a[1]=4
old a[2]=5
new a[2]=6
```

引用

■和指针的区别

- 不存在空引用。引用必须连接到一块合法的内存。
- 一旦引用被初始化为一个对象，就不能被指向到另一个对象。指针可以在任何时候指向到另一个对象。
- 引用必须在创建时被初始化为一个对象。指针可以在初始化时置空，之后再指向对象。

反思：为什么要“引用”？

- The easiest way to think about a reference is as a fancy pointer. You never have to wonder whether it's been initialized(the compiler enforces it) and how to dereference it(the compiler does it).

——《Thinking in C++》

■ 引用的优势：更灵活地支持运算符重载

- 运算符重载是为了写法上的方便，如重载‘+=’，使得类的两个对象可以像数字一样相加：a+=b
- 但如果没有引用，且在值传递对象不合适的情况下，需要使用指针进行重载，即使用者需要进行显式的取地址操作：a+=&b，这是不方便的

- 引用的特性：创建时必须初始化、初始化后便不能指向其他对象，不存在空引用，相对指针更安全

思考：如何像基本类型一样的操作？

- 基本类型和自定义类型的差别是什么？
- 基本类型： **int, long, char, double, float**
- 自定义类型

```
class Test {  
    int data[100];  
public:  
    void setdata(const int*);  
    const int* getdata();  
    void operation1(int);  
};
```

- 如何像基本类型的**操作(功能)**一样，实现对自定义类型的类似**操作(功能)**

类的运算符重载

■ 为什么需要类运算符重载?

- 用户自定义类，没有对常用的运算符进行定义，比如想要表示两个类对象相加，无法采用a+b这种方式。
- 可以采取定义一个add函数的方式，解决这种问题。

```
A add(A a, A b) {  
    return A(a.data + b.data);  
}
```

- 但这种实现方式，在调用的时候，会和基础类型差别很大，缺少**编程的一致性**。需要过多地区分自定义类和基础类型，调用起来也不方便。
- 因此，我们引入**运算符重载**

运算符重载

■ 运算符重载需要按规则声明执行该运算的函数

- 例如 + 对应 operator+

■ 运算重载一般有两种方式（注意参数不同）

- 全局函数的运算符重载

A operator+(A a, A b) {...}

- 成员函数的运算符重载

```
class A{  
    int data;  
public:  
    A operator+(A b) {...};  
};
```

运算符重载

```
#include <iostream>
using namespace std;
class A {
public:
    int data;
    A(int i) { data = i; }
    A& operator+=(A& a) { data += a.data; return *this;}
    // A operator+(A& a) {
    //     A new_a(data + a.data);
    //     return new_a;
    // }
};
A operator+(A& a1, A& a2) {
    A new_a(a1.data + a2.data);
    return new_a;
}
int main() {
    A a1(2), a2(3);
    a1 += a2; // 调用operator+=()
    cout << a1.data << endl; // 5
    cout << (a1 + a2).data << endl; // 调用operator+()
    return 0;
}
```

- 同一运算符 (+) 只能采用一种实现 (紫色或蓝色)

可以重载的运算符

■ 双目算术运算符

- + (加), - (减), * (乘), / (除), % (取模)

■ 关系运算符

- ==(等于), != (不等于), < (小于), > (大于),
<=(小于等于), >=(大于等于)

■ 逻辑运算符

- || (逻辑或), && (逻辑与), !(逻辑非)

■ 单目运算符

- + (正), - (负), * (指针), & (取地址)

■ 自增自减运算符

- ++ (自增), -- (自减)

可以重载的运算符

■ 位运算符

- `|` (按位或), `&` (按位与), `~` (按位取反), `^` (按位异或), `<<` (左移), `>>` (右移)

■ 赋值运算符

- `=`, `+=`, `-=`, `*=`, `/=`, `% =`, `&=`, `|=`, `^=`, `<<=`, `>>=`

■ 空间申请与释放

- `new`, `delete`, `new[]`, `delete[]`

■ 其他运算符

- `()` (函数调用), `->` (成员访问), `,` (逗号), `[]` (下标)

前缀与后缀的++、--

■ 前缀运算符重载声明

- `ClassName operator++();`
- `ClassName operator--();`

■ 后缀运算符重载声明

- `ClassName operator++(int dummy);`
 - `++a ⇔ operator++(a)`
 - `a++ ⇔ operator++(a, int)`
- `ClassName operator--(int dummy);`

■ 通过在函数体中没有使用的哑元参数dummy来区分前缀与后缀的同名重载

■ 哑元可以没有变量名，例如

```
int fun(int, int a){ return a/10*10; }
```

++前缀、后缀语义

前缀语义:

`int a = ++b;` // 先完成**b+1**操作，再赋值

后缀语义:

`int a = b++;` // 先完成赋值，再**b+1**操作

前缀运算符++重载示例

```
#include <iostream>
using namespace std;
```

```
class Test {
public:
    int data = 1;
    Test(int d) {data = d;}
    Test& operator++ () {
        ++data;
        return *this;
    }
};
```

```
int main() {
    Test test(1);
    ++test;
    return 0;
}
```

后缀运算符++重载示例

```
#include <iostream>
using namespace std;
```

```
class Test {
public:
    int data = 1;
    Test(int d) {data = d;}
    Test operator++ (int) {
        Test test(data);
        ++data;
        return test;
    }
};
```

这里定义了一个哑元参数int，但是没有任何变量名，所以函数实现中永远不会用到该变量

```
int main() {
    Test test(1);
    test++;
    return 0;
}
```

前缀与后缀的++、--

■也可以使用全局重载的方式

```
#include <iostream>
using namespace std;
```

```
class A {
public:
    int data;
    A() { data = 0; }
    A(int i) { data = i; }
};
```

```
A operator++(A& a) { //前缀
    ++a.data;
    return a;
}
```

```
A operator++(A& a, int) { //哑元, 后缀
    A new_a(a.data);
    ++a.data;
    return new_a;
}
```

```
int main() {
    A a(1);
    cout << (++a).data << endl; // 2
    cout << (a++).data << endl; // 2
    cout << a.data << endl; // 3
    return 0;
}
```

函数运算符 () 重载

- 在自定义类中也可以重载函数运算符(), 它使对象看上去像是一个函数名

```
ReturnType operator() (Parameters) {  
    ...  
}
```

```
ClassName Obj;
```

```
Obj(real_parameters); // 注意不是调用构造函数!  
// → Obj.operator() (real_parameters);
```

函数运算符（）重载示例

```
#include <iostream>
using namespace std;
```

```
class Test {
public:
```

```
    int operator() (int a, int b) {
        cout << "operator() called. " << a << ' ' << b << endl;
        return a + b;
    }
};
```

```
int main() {
    Test sum;
    int s = sum(3, 4); /// sum对象看上去象是一个函数，故也称“函数对象”
    cout << "a + b = " << s << endl;

    int t = sum.operator()(5, 6);
    return 0;
}
```

数组下标运算符 [] 重载

- 函数声明形式

返回类型 **operator[]** (参数);

- 如果返回类型是引用，则数组运算符调用可以出现在等号左边，接受赋值，即

Obj[index] = value;

- 如果返回类型不是引用，则只能出现在等号右边

Var = Obj[index];

注意：这里**Obj**是一个对象，而不是一个数组

数组下标运算符重载示例

```
#include <iostream>      // cout
#include <cstring>       // strcmp
using namespace std;

char task_name[4][10] = { "vision", "nlp", "audio", "control"};

class AgentScores {
    int scores[4];
    int error_score;
public:
    int& operator[] (const char* name) // 字符串作下标
    {
        for (int i = 0; i < 4; i++) {
            if (strcmp(task_name[i], name) == 0)
                return scores[i];
        }
        return error_score; // 没有匹配到字符串
    }
};
```

注意返回值

数组下标运算符重载示例

```
/// 关于数组下标运算符重载的测试
int main()
{
    AgentScores agent1;
    agent1["nlp"] = 95;
    agent1["vision"] = 88;
    cout    << "NLP Task Score: "
            << agent1["nlp"] << endl;

    return 0;
}
```

运行输出结果

NLP Task Score: 95

只能成员函数重载的运算符

■=, [], (), -> 只能通过成员函数来重载

```
class cls {  
public:  
    int data;  
    cls(int i) : data(i) {}  
};  
cls& operator[](cls& c1, cls& c2) { return c2; }
```

编译错误: error: 'cls& operator[](cls&, cls&)' must be a **nonstatic member function**

只能成员函数重载的运算符

■ 为什么这么做?

- 当没有自定义operator=时，编译器会自动合成一个默认版本的赋值操作
- 在类内定义operator=，编译器则不会自动合成
- 如果允许使用全局函数重载，可能会对是否自动合成产生干扰

假设能全局重载的例子

cls.h:

```
class Cls {  
public:  
    int data;  
    Cls(int _data): data(_data) {}  
};
```

//main.cpp:

```
int main(){  
    Cls a, b(3);  
    a = b; //?  
    return 0;  
}
```

//func.cpp:

```
Cls& operator=(Cls &a, Cls b)  
{  
    cout << "operator=" << endl;  
    a.data = b.data;  
    return a;  
}
```

调用自动合成运算？还是全局重载运算？

C++标准禁止了operator=的全局重载

对象输入输出 —— 流运算符重载

- 用户自定义的类，虽然可以像内置类型那样定义变量（对象），但想要使用流运算符输入、输出对象，则还需要为类定义流运算符重载。

- 如：

```
Test obj;
```

```
cin >> obj;
```

```
...
```

```
cout << obj;
```

```
...
```

流运算符重载函数的声明

```
istream& operator>> (istream& in, Test&  
dst );
```

```
ostream& operator<< (ostream& out, const  
Test& src );
```

- 函数名为：**operator>>** 和 **operator<<**
- 不修改**istream**和**ostream**类的情况下，只能使用全局函数重载
- 返回值为：**istream&** 和 **ostream&**，均为引用
- 参数分别：流对象的引用、目标对象的引用。对于输出流，目标对象一般是常量引用。

流运算符重载示例

```
#include <iostream>
using namespace std;

class Test {
    int id;
public:
    Test(int i) : id(i) { cout << "obj_" << id << " created\n"; }

    friend istream& operator>> (istream& in, Test& dst);
    friend ostream& operator<< (ostream& out, const Test& src);
};

istream& operator>> (istream& in, Test& dst) {
    in >> dst.id;
    return in;
}

ostream& operator<< (ostream& out, const Test& src) {
    out << src.id << endl;
    return out;
}

int main() {
    Test obj(1);
    cout << obj; // operator<<(cout,obj)
    cin >> obj;   // operator>>(cin,obj)
    cout << obj;
    return 0;
}
```

思考题：
为什么形参和返回值都是引用？

为什么返回值要引用？

```
ostream& operator<< (ostream& out, const Test& src) {  
    out << src.id << endl;  
    return out;  
}
```

//测试代码

```
cout << obj << obj2 << obj3 << endl;
```

//等价于

```
ostream& out = operator<<(cout, obj); //return cout;  
ostream& out1 = operator<<(out, obj2); //return cout;  
ostream& out2 = operator<<(out1, obj3); //return cout;
```

课后阅读

■ 《C++编程思想》

- 初始化与清除，第6章
- 运算符重载，第12章

课后练习（不需提交）

■ 不给成员变量初始化会怎样？

- 看下列代码，猜猜运行结果

```
#include <iostream>
using namespace std;
class A
{
public:
    int a;
    int b = 1;
};
A obj1;
int main()
{
    A obj2;
    static A obj3;
    cout << obj1.a << " " << obj1.b << endl;
    cout << obj2.a << " " << obj2.b << endl;
    cout << obj3.a << " " << obj3.b << endl;
    return 0;
}
```

课后练习（不需提交）

■ 提示

- 全局变量和函数静态变量会自动置0
- 局部变量一般不会自动初始化，其值为未定义。具体运行结果可能会和系统、编译器有关。

■ 拓展阅读

- <https://blog.csdn.net/dog250/article/details/107403337>

课后练习（不需提交）

■ 试试运算符重载吧

- 我们知道，int的范围是 $-2^{31} \sim 2^{31}-1$ 。请你实现一个范围更大的Int，定义如下

```
class Int
{
    private:
        int high, low; //实际值为high * 10^8 + low;
};
```

- 尝试重载运算符
 - 可以试试成员函数重载与全局重载两种方法
 - 下一页给出几个最核心的几个运算符，更多的查看课上运算符列表

精简列表

■ 简单难度:

- 双目算术运算符
 - + (加), -(减)
- 关系运算符
 - ==(等于), < (小于)

■ 中等难度

- 单目运算符 - (负号)

■ 高级难度

- (注意返回值是否需要引用, 有什么区别?)
- 自增运算符 (前置和后置) 两种 ++ (自增)
- 赋值运算符 =, +=
- 流运算符 (cout) <<, >>

结束